

ESPECIFICACIÓN DE ALCANCE DEL SISTEMA

Sistema de Gestión y Simulación de Torneos de eSports por Consola (CLI)

Documento: Historia de Alcance y Requerimientos Arquitectura: Modular Python (main.py / funciones.py) Estado: Aprobado para Desarrollo

1. PROBLEMA

El crecimiento acelerado de la industria de los deportes electrónicos (eSports) ha generado la necesidad de organizar competencias dinámicas de forma rápida y automatizada. En entornos educativos, ligas amateur y eventos de prueba de concepto, la gestión manual de enfrentamientos, cómputo de puntajes, tablas de posiciones y estadísticas agregadas resulta altamente ineficiente y propensa a errores de cálculo.

Actualmente, la falta de herramientas livianas e independientes para simular torneos bajo reglas específicas de dominio provoca demoras operativas. Los organizadores requieren un sistema que no dependa de complejas interfaces gráficas ni bases de datos pesadas, sino que funcione directamente sobre una interfaz de línea de comandos (CLI), garantizando rapidez en la carga de datos, aleatoriedad en el rendimiento de los participantes y transparencia en la determinación de ganadores.

DIAGNÓSTICO DE LA NECESIDAD

Se necesita una solución de software modular estructurada en Python que permita definir equipos, simular enfrentamientos al mejor de 3 rondas mediante generación aleatoria validada, estructurar la información en matrices y tuplas, y presentar métricas consolidadas (promedios y puntajes máximos) de forma interactiva.

Matriz de Ineficiencias Actuales vs. Solución Propuesta

ASPECTO	PROCESO MANUAL / TRADICIONAL	SOLUCIÓN CLI AUTOMATIZADA
Cómputo de Puntajes	Registro manual en planillas, propenso a inconsistencias aritméticas.	Procesamiento matricial automático mediante rutinas en Python.
Simulación de Enfrentamientos	Dependiente de arbitraje presencial prolongado para cada serie.	Generación de resultados aleatorios validados bajo la regla "Mejor de 3".
Búsqueda de Estadísticas	Revisión hoja por hoja para auditar el desempeño de un equipo.	Función dedicada con búsqueda y despliegue tabular directo.
Arquitectura de Código	Programas monolíticos y difíciles de mantener.	Diseño modular dividido estrictamente en main.py y funciones.py.

2. OBJETIVO DEL PROYECTO

Desarrollar un sistema por consola (CLI) en el lenguaje de programación Python que automatice íntegramente la administración, simulación y análisis estadístico de un torneo de eSports. El sistema se diseñará bajo el paradigma de programación estructurada y modular, garantizando un código limpio, legible y eficiente.

Objetivos Específicos

- **Estructuración Modular:** Dividir la lógica del sistema estrictamente en dos archivos: `main.py` (controlador de flujo, menú interactivo y entrada/salida) y `funciones.py` (lógica de negocio, funciones lambda, transformaciones matriciales y validaciones).
- **Simulación Estocástica Controlada:** Implementar la librería `random` para la generación de puntajes aleatorios en cada ronda, aplicando restricciones formales para asegurar el cumplimiento del formato "Mejor de 3" (un ganador requiere ganar al menos 2 rondas).
- **Gestión de Datos Multidimensionales:** Utilizar matrices para almacenar la evolución del torneo, tablas de estadísticas e indicadores de promedios por equipo, respaldando los listados fijos e iniciales mediante tuplas inmutables.
- **Normalización y Validación de Entrada:** Garantizar que la incorporación de nuevos equipos cumpla con criterios de formato (normalización de cadenas y longitud mínima) antes de su integración a las estructuras de datos.
- **Presentación Dinámica:** Formatear todas las salidas por pantalla mediante `f-strings` para ofrecer una interfaz visual clara, ordenada y profesional en consola.

META DEL SISTEMA	INDICADOR DE ÉXITO	MECANISMO DE VERIFICACIÓN
Modularidad Completa	100% de funciones auxiliares y de simulación alojadas en <code>funciones.py</code> .	Inspección de código fuente y llamadas desde <code>main.py</code> .
Regla de Dominio "Mejor de 3"	Cero empates o victorias inválidas (máximo 3 rondas, mínimo 2 victorias).	Validación lógica sobre la matriz de enfrentamientos.
Integridad de Datos	Garantizar nombres normalizados y consistencia en dimensiones matriciales.	Ejecución de validaciones previas al almacenamiento.

3. DATOS DEL SISTEMA Y ARQUITECTURA DE SOFTWARE

El sistema adopta un diseño modular estricto para separar la interacción con el usuario del procesamiento lógico de los datos. A continuación se detalla la responsabilidad de cada componente informático y las estructuras de datos empleadas.

3.1. Estructura de Archivos y Responsabilidades

ARCHIVO	ROL EN EL SISTEMA	COMPONENTES E INVOCACIONES
main.py	Punto de entrada del programa, orquestador del menú interactivo y gestor de parámetros iniciales.	- Definición de entradas: Cantidad máxima de equipos y rondas. - Bucle principal (Menú de opciones). - Definición e inicialización de matrices dinámicas. - Salida por consola formateada mediante f-strings.
funciones.py	Biblioteca de lógica de negocio, funciones de procesamiento, filtrado y validación.	- Uso de funciones lambda para cálculos veloces. - VisualizarEquipos() - AgregarEquipo() - SimularTorneo() - BuscarEquipo()

3.2. Estructuras de Datos Utilizadas

- Tuplas:** Se utilizan para almacenar datos inmutables, especificando los equipos iniciales elegibles y las configuraciones fijas de puntajes por ronda.
- Matrices (Listas Anidadas):** Constituyen el motor de almacenamiento principal del torneo:
 - Matriz del Torneo:* Contiene el registro detallado de puntos obtenidos por equipo en cada ronda, el puntaje total acumulado y su posición final.
 - Matriz de Estadísticas/Promedios:* Almacena el cálculo auxiliar de promedios por ronda, puntajes máximos alcanzados e historial consolidado.

3.3. Mapa del Menú Interactivo

```
=====
      MENU PRINCIPAL - TORNEO ESPORTS CLI
=====
1. Ver equipos iniciales
2. Agregar un equipo
3. Simular torneo
4. Buscar equipo
5. Puntaje máximo
6. Puntaje promedio por equipo
7. Fin
=====
```

OPCIÓN DEL MENÚ	FUNCIÓN ASOCIADA	DESCRIPCIÓN OPERATIVA
1. Ver equipos iniciales	VisualizarEquipos()	Muestra los equipos disponibles en la tupla inicial mediante f-strings.
2. Agregar un equipo	AgregarEquipo()	Pide el nombre del equipo, valida longitud/normalización e integra a la lista.
3. Simular torneo	SimularTorneo()	Genera los puntajes aleatorios por ronda, computa la matriz final y determina la posición.
4. Buscar equipo	BuscarEquipo()	Filtra la matriz de estadísticas e imprime la ficha de rendimiento del equipo solicitado.
5. Puntaje máximo	Lógica auxiliar / Lambda	Calcula y despliega el equipo con la mayor cantidad de puntos en una ronda o torneo.
6. Puntaje promedio por equipo	Matriz Auxiliar / Lambda	Calcula $P_{prom} = \Sigma Puntos / Total Rondas$ por cada equipo participante.
7. Fin	Control de Flujo	Termina la ejecución del bucle principal y cierra el sistema.

4. REGLAS DEL TORNEO Y ALGORITMO DE SIMULACIÓN

El torneo se desarrolla bajo un esquema competitivo estructurado por rondas. El motor de simulación garantiza dinamismo mediante la aleatoriedad, pero sujeto a restricciones lógicas estrictas para respetar las reglas formales de la competencia de eSports.

4.1. Definición del Enfrentamiento "Mejor de 3"

Cada serie entre dos participantes o cada enfrentamiento directo de ronda se dispute en un formato de máximo tres partidas ("Mejor de 3"). Un equipo es declarado vencedor del cruce únicamente si logra alcanzar **2 rondas/partidas ganadas**.

REGLA DE VALIDACION ALGORÍTMICA

Si un equipo gana las primeras 2 rondas de forma consecutiva (2-0), no es necesario simular la tercera ronda. Si hay empate 1-1, obligatoriamente se simula la tercera ronda para definir al ganador (2-1). No se permiten resultados con 0 rondas ganadas en victoria ni empates al finalizar el enfrentamiento.

4.2. Cómputo Matemático y Formulaciones

Para la construcción de las matrices de puntaje y la matriz de promedios se emplean los siguientes modelos matemáticos directos:

Puntaje acumulado total por equipo i :

$$P_{total}(i) = \sum_{j=1}^R P(i, j)$$

Donde $P(i, j)$ representa los puntos obtenidos aleatoriamente por el equipo i en la ronda j , y R es el límite máximo de rondas configurado en el `main.py`.

Puntaje promedio por equipo:

$$P_{prom}(i) = P_{total}(i) / R$$

4.3. Flujo Lógico de Simulación de Enfrentamiento

PASO	ACCIÓN	CRITERIO / VALIDACIÓN
Paso 1	Inicialización de contadores de victorias.	<code>victorias_A = 0, victorias_B = 0.</code>
Paso 2	Simulación de partida mediante <code>random.randint()</code> .	Asignación de puntos aleatorios a cada bando.
Paso 3	Evaluación de ganador de la partida individual.	Incremento de victoria al equipo con mayor puntaje en la sub-ronda.
Paso 4	Verificación de condición de parada ("Mejor de 3").	¿Algún equipo llegó a 2 victorias? Si SI, finaliza la serie. Si NO, repetir Paso 2.
Paso 5	Registro en la Matriz del Torneo.	Cálculo de total de puntos acumulados y asignación de posición global.

5. REGLAS DE DOMINIO, VALIDACIONES Y OMISIONES

Las reglas de dominio establecen los límites operativos y de integridad de datos que rigen el comportamiento del software.

5.1. Reglas de Dominio de Datos

- **Normalización de Nombres:** Todos los nombres de equipos ingresados mediante la función `AgregarEquipo()` deben ser transformados a formato estandarizado (por ejemplo, eliminación de espacios innecesarios al inicio/final con `.strip()` y capitalización de palabras).
- **Longitud Mínima:** El nombre de un nuevo equipo debe contar obligatoriamente con un mínimo de caracteres (ejemplo: mínimo 3 caracteres alfabéticos) para ser admitido.
- **Inmutabilidad Base:** Los equipos por defecto propuestos por el sistema residen en tuplas, impidiendo su alteración accidental durante la simulación.
- **Uso de Funciones Lambda:** Se exige la inclusión de expresiones `lambda` en `funciones.py` para tareas inmediatas de filtrado, ordenamiento de posiciones en la matriz y obtención de valores máximos.

5.2. Delimitación de Alcance: Excepciones y Omisiones Explicitadas

Con el objetivo de mantener la simplicidad pedagógica, la ligereza del sistema y enfocar el desarrollo en el manejo de matrices, tuplas y lógica algorítmica pura, se han establecido las siguientes omisiones formales en el diseño:

OMISIÓN FORMAL DE CONTROL DE EXCEPCIONES

No se utiliza bloques `try / catch` (o `try / except` en Python) para el manejo de errores en tiempo de ejecución. El control de flujo asume que las entradas numéricas del menú e inputs de configuración seguirán los tipos de datos requeridos, enfocando la validación en condicionales lógicos analíticos (`if / else`) para strings y rangos.

ELEMENTO EXCLUIDO	JUSTIFICACIÓN TÉCNICA / SUSTITUTO LÓGICO
Manejo de Excepciones (<code>try / except</code>)	Se sustituye por validaciones explícitas mediante condicionales <code>if/else</code> sobre cadenas y rangos de datos.
Persistencia en Archivos / DBs	El alcance es en memoria volátil (RAM) mientras dure la ejecución del programa en consola.
Interfaz Gráfica (GUI/Web)	El sistema es 100% en interfaz de consola CLI, utilizando <code>f-strings</code> para dar formato tabular.

5.3. Resumen de Estructura de Salida por Consola

A continuación se muestra una representación del formato esperado para la matriz final resultante procesada por `SimularTorneo()`:

```
+-----+
|                                     |
|               TABLA FINAL DEL TORNEO               |
|-----+-----+-----+-----+-----+
| POS| EQUIPO          | RONDA 1 | RONDA 2 | PUNTAJE TOTAL | PROM |
|-----+-----+-----+-----+-----+
|  1 | Vipers eSports  |    85   |    92   |       177     | 88.50 |
|  2 | Cyber Dragons   |    78   |    88   |       166     | 83.00 |
|  3 | Alpha Knights   |    90   |    70   |       160     | 80.00 |
+-----+-----+-----+-----+-----+
```